

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №6

Тема «Накладання текстур»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 23:

1. Змініть фігуру.
2. Змініть текстуру.
3. Додайте трансформації (наприклад обернути на 45 градусів) фігури із текстурою.
4. Опишіть загальний алгоритм накладання текстури.
5. Наведіть коментарі до коду.
6. Внесіть індивідуальні зміни до коду.

РЕЗУЛЬТАТ

Код

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

// Індивідуальна зміна: Винесення констант
const float INITIAL_ROTATION_ANGLE = 45.0f; // Початковий кут обертання
const char *TEXTURE_PATH = "box_texture.png"; // Шлях до зміненої
текстури

// Функція для завантаження текстури (залишена без змін, але не
використовується в main)
unsigned int loadTexture(const char *path)
{
    unsigned int textureID;
    glGenTextures(1, &textureID);
    int width, height, nrChannels;
    // stbi_load звільнить пам'ять, якщо поверне NULL, але
    stbi_image_free(data) в else-гілці не потрібен
    unsigned char *data = stbi_load(path, &width, &height, &nrChannels,
0);
    if (data)
    {
        GLenum format;
        if (nrChannels == 1)
```

```

        format = GL_RED;
    else if(nrChannels == 3)
        format = GL_RGB;
    else if(nrChannels == 4)
        format = GL_RGBA;

    // Встановлення параметрів текстури
    glBindTexture(GL_TEXTURE_2D, textureID);
    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
format, GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);

    // Встановлення параметрів фільтрації
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR); // виправлено фільтр для міпмапів
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);

    stbi_image_free(data);
}
else
{
    // Помилка завантаження
    fprintf(stderr, "Texture failed to load at path: %s\n", path);
    stbi_image_free(data); // звільнення, якщо data != NULL, але
тут data = NULL
}
return textureID;
}

int main()
{
    // Ініціалізація GLFW
    if (!glfwInit())
    {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }

    // Налаштування версії OpenGL 3.3 Core Profile
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

    // Створення вікна
    GLFWwindow *window = glfwCreateWindow(800, 600, "Texture Example",
NULL, NULL);
    if (!window)
    {
        fprintf(stderr, "GLFW window creation failed\n");
        glfwTerminate();
        return -1;
    }
    glfwMakeContextCurrent(window);

```

```

// Ініціалізація GLEW
if (glewInit() != GLEW_OK)
{
    fprintf(stderr, "GLEW initialization failed\n");
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}

// Увімкнення тесту глибини
glEnable(GL_DEPTH_TEST);
glViewport(0, 0, 800, 600);

// Код вершинного шейдера
const char *vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;
    layout (location = 1) in vec2 aTexCoord;

    out vec2 TexCoord;

    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;

    void main() {
        gl_Position = projection * view * model * vec4(aPos, 1.0);
        TexCoord = aTexCoord;
    }
)";

// Код фрагментного шейдера
const char *fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;

    in vec2 TexCoord;
    uniform sampler2D texture1; // Семплер для доступу до даних
текстури

    void main() {
        // Отримання кольору з текстури за координатами TexCoord
        FragColor = texture(texture1, TexCoord);
    }
)";

// Компіляція та лінування шейдерів
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);
GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);
GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);

```

```

glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

// Дані вершин для квадрата (4 вершини, 2 трикутники).
// Формат: X, Y, Z, U, V
GLfloat vertices[] = {
    // Позиція          // Текстурні координати
    0.5f,  0.5f, 0.0f,  1.0f, 1.0f, // Верхній правий (0)
    0.5f, -0.5f, 0.0f,  1.0f, 0.0f, // Нижній правий (1)
    -0.5f, -0.5f, 0.0f,  0.0f, 0.0f, // Нижній лівий (2)
    -0.5f,  0.5f, 0.0f,  0.0f, 1.0f  // Верхній лівий (3)
};

// Індеси для формування двох трикутників (квадрата)
GLuint indices[] = {
    0, 1, 3, // Перший трикутник
    1, 2, 3  // Другий трикутник
};

// Створення VAO, VBO, EBO
GLuint VAO, VBO, EBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glGenBuffers(1, &EBO);

glBindVertexArray(VAO);

// VBO: Завантаження даних вершин
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

// EBO: Завантаження даних індексів
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

// Атрибут 0: Позиція (3 float)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 *
sizeof(GLfloat), (GLvoid *)0);
glEnableVertexAttribArray(0);

// Атрибут 1: Текстурні координати (2 float, зміщення 3 *
sizeof(GLfloat))
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 *
sizeof(GLfloat), (GLvoid *) (3 * sizeof(GLfloat)));
glEnableVertexAttribArray(1);

glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);

// Завантаження текстури (використовуємо спрощену логіку з main,
але з новою назвою)
unsigned int texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);

// Встановлення параметрів обгортання

```

```

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);

// Встановлення параметрів фільтрації
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

// Індивідуальна зміна: stbi_set_flip_vertically_on_load(true) тут
вже є
stbi_set_flip_vertically_on_load(true);

// Завантаження даних зображення (використовуємо нову змінну
TEXTURE_PATH)
int width, height, nrChannels;
unsigned char *data = stbi_load(TEXTURE_PATH, &width, &height,
&nrChannels, 0);

if (data)
{
    GLenum format;
    if (nrChannels == 1)
        format = GL_RED;
    else if(nrChannels == 3)
        format = GL_RGB;
    else if(nrChannels == 4)
        format = GL_RGBA;

    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
format, GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);
    stbi_image_free(data);
}
else
{
    fprintf(stderr, "Failed to load texture: %s\n", TEXTURE_PATH);
    stbi_image_free(data);
}

// Основний цикл рендерингу
while (!glfwWindowShouldClose(window))
{
    // Очищення буферів (темно-сірий колір)
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Обчислення матриць
    glm::mat4 model = glm::mat4(1.0f);

    // Індивідуальна зміна/Трансформація: Обертання
    // Обертання навколо осі Y, кут збільшується з часом
    float rotationAngle = INITIAL_ROTATION_ANGLE +
(float)glfwGetTime() * 50.0f; // 50 градусів/сек
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f)); //
Залишаємо в центрі
    model = glm::rotate(model, glm::radians(rotationAngle),
glm::vec3(0.0f, 1.0f, 0.0f)); // Обертання по Y

```

```

        glm::mat4 view = glm::lookAt(glm::vec3(0.0f, 0.0f, 3.0f),
glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f)); // Позиція
камери
        glm::mat4 projection = glm::perspective(glm::radians(45.0f),
800.0f / 600.0f, 0.1f, 100.0f); // Матриця проєкції

        // Передача uniform-змінних у шейдер
        GLuint modelLoc = glGetUniformLocation(shaderProgram, "model");
        GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
        GLuint projectionLoc = glGetUniformLocation(shaderProgram,
"projection");

        glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
glm::value_ptr(projection));

        glUseProgram(shaderProgram);

        // Прив'язка текстури
        glBindTexture(GL_TEXTURE_2D, texture);

        // Рендеринг квадрата
        glBindVertexArray(VAO);
        glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0); // 6
індексів для двох трикутників
        glBindVertexArray(0);

        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    // Очищення ресурсів
    glDeleteVertexArrays(1, &VAO);
    glDeleteBuffers(1, &VBO);
    glDeleteBuffers(1, &EBO);
    glfwDestroyWindow(window);
    glfwTerminate();
    return 0;
}

```